03 — Natural Language to SQL / HANA AI Query (Deep Dive)

Module purpose: lets a user ask a plain-English question and get back a pandas DataFrame — the LLM writes HANA SQL against a schema you supply, with two independent safety layers so it can never run anything except `SELECT`. Five files, each a clean stage of the pipeline.

File 1 — `schema_inspector.py`

```
def get_table_schema(conn, table_name: str, schema_name: str = None) -> dict:
    table_name = table_name.upper()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT COLUMN_NAME, DATA_TYPE_NAME FROM SYS.COLUMNS "     # SYS.COLUMNS = HANA's
        ↪  system catalog view
        "WHERE TABLE_NAME = ? AND SCHEMA_NAME = CURRENT_USER ORDER BY POSITION",
        (table_name,)
    )
    rows = cursor.fetchall()
    columns = [{"name": r[0], "type": r[1]} for r in rows]
    return {"table": table_name, "schema": schema_name or "CURRENT_USER", "columns": columns}

def build_schema_context(conn, tables: list) -> str:
    lines = []
    for entry in tables:
        info = get_table_schema(conn, entry["table"], entry.get("schema")) if
↪   isinstance(entry, dict) \
              else get_table_schema(conn, entry)
        lines.append(f"Table: {info['schema']}.{info['table']}")
        for col in info["columns"]:
            lines.append(f"  - {col['name']:<30} : {col['type']}")     # human-readable,
↪   LLM-friendly text
    return "\n".join(lines)
```

Working explanation: the LLM cannot see your database — `SYS.COLUMNS` is HANA's built-in metadata catalog (every HANA install has it), so querying it gives real, current column names and types with zero guessing. `build_schema_context` turns that into a plain-text block that gets pasted directly into the prompt as `{schema_context}` — this is the entire trick behind "AI that knows my schema": you're not fine-tuning anything, you're just showing the model the DDL-equivalent info at request time (a lightweight form of retrieval/grounding).

File 2 — `sql_generator.py`

```
SQL_PROMPT = ChatPromptTemplate.from_template("""
You are an expert SAP HANA SQL developer.
Rules:
- Write ONLY a single SELECT statement. No INSERT/UPDATE/DELETE/DROP/DDL.
- Use only tables/columns in the schema below.
- For "top N", use: FETCH FIRST N ROWS ONLY   (HANA syntax, not LIMIT or SELECT TOP)
Schema: {schema_context}
Question: {user_query}
SQL Query:
""")
```

```python
def generate_sql(user_query, schema_context, llm) -> str:
    chain = SQL_PROMPT | llm | StrOutputParser()          # LangChain Expression Language
↪  (LCEL) pipe
    raw = chain.invoke({"schema_context": schema_context, "user_query": user_query})
    sql = re.sub(r"```(?:sql)?", "", raw, flags=re.IGNORECASE).strip().strip("`")   #
↪  defensive cleanup
    return sql
```

Working explanation: `SQL_PROMPT | llm | StrOutputParser()` is LangChain's LCEL — each `|` feeds the output of the left side into the right side: the prompt template renders with your variables, the rendered prompt goes to the LLM, and `StrOutputParser` extracts the plain string from the LLM's structured response object. The regex cleanup exists because LLMs sometimes wrap SQL in "'sql fences even when told not to — never assume instruction-following is perfect.

File 3 — `sql_executor.py`

```python
def execute_sql(conn, sql: str) -> pd.DataFrame:
    first_word = sql.strip().split()[0].upper()
    if first_word != "SELECT":                    # <- independent safety check #2 (not just
    ↪  prompt wording)
        raise ValueError(f"Only SELECT queries are allowed. Got: '{first_word}'")
    cursor = conn.cursor()
    try:
        cursor.execute(sql)
        rows, cols = cursor.fetchall(), [d[0] for d in cursor.description]
        return pd.DataFrame(rows, columns=cols)
    except Exception as e:
        raise Exception(f"SQL execution failed: {e}\nSQL:\n{sql}")   # re-raise with the
        ↪  failing SQL attached
    finally:
        cursor.close()
```

Working explanation: this is the **critical security boundary** of the whole module — even if the prompt's SELECT-only rule fails (prompt injection, model mistake), this function independently inspects the actual SQL string's first token before it ever reaches `cursor.execute()`. Errors are re-raised with the failing SQL attached, which is invaluable for debugging bad generations without needing separate logging.

File 4 — `query_agent.py` (wires 1-3 together)

```python
class QueryAgent:
    def __init__(self, conn, tables: list, llm=None):
        self.conn = conn
        self.llm = llm or get_llm()
        self.schema_context = build_schema_context(conn, tables)   # computed ONCE at
        ↪  construction

    def ask(self, user_query: str) -> pd.DataFrame:
        sql = generate_sql(user_query, self.schema_context, self.llm)
        return execute_sql(self.conn, sql)

    def get_schema(self) -> str:
```

```
        return self.schema_context     # useful for debugging what the LLM actually sees
```

Working explanation: schema is fetched **once** in `__init__`, not on every `.ask()` call — a deliberate performance choice, since table structure rarely changes within a session. If your schema changes at runtime, you'd need to re-instantiate `QueryAgent` or add an explicit refresh method.

File 5 — `query_with_summary.py` (adds a second LLM call for a plain-English answer)

```
SUMMARY_PROMPT = ChatPromptTemplate.from_template("""
The user asked: "{user_query}"
The SQL query run was: {sql}
The result has {row_count} rows. Sample: {sample_data}
Write a concise answer. Do not mention SQL or technical details.
""")

def ask_with_summary(conn, tables, user_query, llm=None) -> dict:
    llm = llm or get_llm()
    schema_context = build_schema_context(conn, tables)
    sql = generate_sql(user_query, schema_context, llm)
    df = execute_sql(conn, sql)
    sample = df.head(20).to_markdown(index=False) if not df.empty else "No results found."
    summary_chain = SUMMARY_PROMPT | llm | StrOutputParser()
    summary = summary_chain.invoke({"user_query": user_query, "sql": sql, "row_count":
↪   len(df), "sample_data": sample})
    return {"sql": sql, "data": df, "summary": summary}
```

Working explanation: two separate LLM calls happen here — one to generate SQL, one to summarize the resulting data in plain English — because mixing both jobs into a single prompt tends to produce worse results at both tasks than specializing each call. `.head(20)` caps the sample fed back to the LLM so huge result sets don't blow the token budget.

Key Points

- **Two independent safety layers**: prompt instruction (soft) + `execute_sql`'s first-word check (hard) — never rely on prompt wording alone for security.
- `SYS.COLUMNS` is how the LLM "sees" your schema — no fine-tuning, just runtime context injection.
- HANA SQL dialect quirk: `FETCH FIRST N ROWS ONLY` (also accepts `SELECT TOP N`), **not** `LIMIT`.
- LCEL pipe syntax (`prompt | llm | parser`) chains transformations left to right — the standard modern LangChain pattern, replacing older `LLMChain` class-based chains.
- Schema context is computed once per `QueryAgent` instance, not per query — a performance/cost tradeoff.
- `query_with_summary` demonstrates **prompt specialization**: one call per distinct job (SQL gen vs. summarization) usually beats one prompt trying to do both.

Interview Q&A

Q: Why not just trust the LLM's prompt instructions to only generate SELECT statements? A: LLMs can misfollow instructions or be prompt-injected via user input; defense in depth means validating the actual generated SQL string independently before it ever touches the database.

Q: What would you change to support JOINs across multiple tables? A: Pass multiple table names (or explicit `{"table":..., "schema":...}` dicts) into `build_schema_context` — the LLM sees every column across all tables in one prompt and can write the JOIN itself; no extra code

changes needed since the prompt/schema mechanism is already generic.

Q: Why does `sql_executor.py` re-raise exceptions with the SQL attached instead of just logging and returning None? A: Silently swallowing DB errors hides bugs; re-raising with context (`SQL execution failed: ...\nSQL:\n...`) gives the caller (and any error monitoring) everything needed to diagnose a bad generation immediately.

Q: How would you reduce the risk of the LLM generating an extremely expensive query (full table scan on a huge table)? A: Add explicit guardrails in the prompt (mandate `FETCH FIRST N ROWS ONLY`), enforce a query timeout at the driver/connection level, and/or run generated queries against a read replica with resource governance limits rather than the primary transactional database.